Vel Tech Group of Institutions

Name: Gontu Yaswanth
Email: vtu33434@veltech.edu.in
Roll no: Vtu33434
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 8_COD

Attempt : 1
Total Mark : 50
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Sanjay is writing a program to manage an undirected graph using an adjacency list representation. Users will initially input the vertices to create the edges. The program will display the adjacency list of the graph.

Assist Sanjay in creating the program.

Input Format

The input consists of 5 nodes.

Edge information: ab ae bc bd be cd de.

Output Format

The output prints the adjacency list representation of the graph.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 0 1 2 3 4

Output: vertex 0: head -> 4 -> 1
vertex 1: head -> 4 -> 3 -> 2 -> 0
vertex 2: head -> 3 -> 1
vertex 3: head -> 4 -> 2 -> 1
vertex 4: head -> 3 -> 1 -> 0

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Structure for a node in the adjacency list
struct Node {
    int data;
    struct Node* next;
};

// Structure for the adjacency list
struct AdjList {
    struct Node* head;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to add an edge (Undirected)
void addEdge(struct AdjList* adj, int src, int dest) {
    // Add dest to src
    struct Node* newNode = createNode(dest);
    newNode->next = adj[src].head;
```

```
    adj[src].head = newNode;

    // Add src to dest
    newNode = createNode(src);
    newNode->next = adj[dest].head;
    adj[dest].head = newNode;
}

int main() {
    int vertices[5];
    struct AdjList adj[5];

    // Initialize adjacency lists
    for (int i = 0; i < 5; i++) {
        if (scanf("%d", &vertices[i]) != 1) return 0;
        adj[i].head = NULL;
    }

    // Hardcoded edges based on problem statement:
    // a=0, b=1, c=2, d=3, e=4 (0-1, 0-4, 1-2, 1-3, 1-4, 2-3, 3-4)
    addEdge(adj, 0, 1); // ab
    addEdge(adj, 0, 4); // ae
    addEdge(adj, 1, 2); // bc
    addEdge(adj, 1, 3); // bd
    addEdge(adj, 1, 4); // be
    addEdge(adj, 2, 3); // cd
    addEdge(adj, 3, 4); // de

    // Print the adjacency list
    for (int i = 0; i < 5; i++) {
        int v = vertices[i];
        printf("vertex %d: head", v);
        struct Node* temp = adj[v].head;
        while (temp) {
            printf(" -> %d", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }

    return 0;
}
```

Status : Wrong **Marks : 0/10**

2. Problem Statement

Sophia is designing a transportation network for a city with multiple interconnected districts. Each district is represented as a vertex, and every road connecting two districts is represented as a directed edge in a graph.

To evaluate connectivity, Sophia needs a program that performs a Breadth-First Search (BFS) traversal of the city's network starting from a specified source district, and outputs the order in which districts are visited.

Write a program to ensure that Sophia can perform the BFS traversal correctly and view the order of visited districts.

Input Format

The first line of input consists of two space-separated integers, v, and e, representing the total number of cities and the number of connections between cities.

The next e lines each contain two space-separated integers a and b, representing a directed edge from city a to city b.

The last line consists of a single integer s, representing the source city from which BFS traversal should start.

Output Format

The output prints the order in which the cities are visited during the BFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6 7
0 1
0 2

1 3
1 4
2 4
3 5
4 5
0

Output: 0 1 2 3 4 5

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

// BFS Function
void bfs(int adj[MAX][MAX], int v, int startNode) {
    int visited[MAX] = {0}; // Track visited cities
    int queue[MAX];         // Queue for BFS
    int front = 0, rear = 0;

    // Visit the source city
    visited[startNode] = 1;
    queue[rear++] = startNode;

    while (front < rear) {
        // Dequeue a city and print it
        int current = queue[front++];
        printf("%d ", current);

        // Check all potential connections from the current city
        for (int i = 0; i < v; i++) {
            if (adj[current][i] == 1 && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
    printf("\n");
}

int main() {
```

```
    int v, e, s;
    int adj[MAX][MAX] = {0}; // Initialize adjacency matrix with 0

    // Read number of cities and connections
    if (scanf("%d %d", &v, &e) != 2) return 0;

    // Read directed edges
    for (int i = 0; i < e; i++) {
        int a, b;
        scanf("%d %d", &a, &b);
        if (a < v && b < v) {
            adj[a][b] = 1; // Directed edge from a to b
        }
    }

    // Read source city
    scanf("%d", &s);

    // Perform BFS
    bfs(adj, v, s);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Liam is developing a social media analytics tool for tracking user interactions. He needs to identify the connected components of users in the network by traversing a directed graph. Each user is represented as a vertex, and the connections between users are represented as directed edges. Liam's task is to implement a Depth-First Search (DFS) algorithm to perform a traversal of the network, starting from a given user's profile.

Can you help Liam by implementing the DFS algorithm to identify all connected users starting from a specified user profile?

Example

Input:

6 6

0 1

0 2

1 2

2 0

2 3

3 3

2

Output:

2 0 1 3

Explanation:

Starting from vertex 2, it visits and marks vertex 2 as visited.

Vertex 2 is printed.

Checking the neighbors of vertex 2: 0 and 3.

Vertex 0 is visited, so DFS is called with vertex 0.

Vertex 0 is visited and printed.

Vertex 1 is visited and printed.

Vertex 3 is visited and printed.

The final output is "2 0 1 3".

Input Format

The first line contains two space-separated integers n and m, representing the total number of users (vertices) and the number of connections (edges) in the network, respectively.

The next m lines contain two space-separated integers u and v, representing a directed connection from user u to user v.

The last line contains a single integer s, representing the ID of the source user profile from which the DFS traversal should start.

Output Format

The output displays a single line containing the user IDs in the order they were visited during the DFS traversal, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6 6
0 1
0 2
1 2
2 0
2 3
3 3
2
Output: 2 0 1 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

// DFS Function
void dfs(int adj[MAX][MAX], int visited[MAX], int v, int current) {
    // Mark current user as visited and print
    visited[current] = 1;
    printf("%d ", current);

    // Explore all outgoing connections from the current user
    for (int i = 0; i < v; i++) {
        // If there's an edge and the user hasn't been visited yet
        if (adj[current][i] == 1 && !visited[i]) {
            dfs(adj, visited, v, i);
        }
```

```
    }
}

int main() {
    int n, m, s;
    int adj[MAX][MAX] = {0}; // Adjacency matrix initialized to 0
    int visited[MAX] = {0};  // Visited array initialized to 0

    // Read number of users (n) and connections (m)
    if (scanf("%d %d", &n, &m) != 2) return 0;

    // Read directed edges
    for (int i = 0; i < m; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        if (u < n && v < n) {
            adj[u][v] = 1;
        }
    }

    // Read the starting user profile (s)
    scanf("%d", &s);

    // Start DFS traversal
    dfs(adj, visited, n, s);
    printf("\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Given a graph with V vertices numbered from 0 to V-1 and a list of edges, determine whether the graph is bipartite or not.

Note: A bipartite graph is a type of graph where the set of vertices can be divided into two disjoint sets, say U and V, such that every edge connects a vertex in U to a vertex in V, there are no edges between vertices within the

same set.

Example:

Input: V = 4, edges[][]= [[0, 1], [0, 2], [1, 2], [2, 3]]

Output: No, the given graph is not Bipartite

Explanation

The graph is not bipartite because no matter how we try to color the nodes using two colors, there exists a cycle of odd length (like 1–2–0–1), which leads to a situation where two adjacent nodes end up with the same color. This violates the bipartite condition, which requires that no two connected nodes share the same color.

Input: V = 4, edges[][] = [[0, 1], [1, 2], [2, 3]]

Output: Yes, the given graph is Bipartite

Explanation:

The given graph can be colored in two colors so, it is a bipartite graph.

Input Format

The first line contains two space-separated integers V and E, representing the number of vertices and edges respectively.

The next E lines each contain two space-separated integers u and v, indicating an undirected edge between vertices u and v.

Output Format

The output print "Yes, the given graph is Bipartite" if the graph is bipartite.

Otherwise, print "No, the given graph is not Bipartite".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4 3
0 1
1 2
2 3

Output: Yes, the given graph is Bipartite

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

// Function to check if the graph is Bipartite using BFS
int isBipartite(int adj[MAX][MAX], int V) {
    int color[MAX];
    for (int i = 0; i < V; i++) color[i] = -1; // -1 means uncolored

    for (int i = 0; i < V; i++) {
        // If the node is not colored, start a BFS
        if (color[i] == -1) {
            int queue[MAX];
            int front = 0, rear = 0;

            color[i] = 1; // Assign first color
            queue[rear++] = i;

            while (front < rear) {
                int u = queue[front++];

                for (int v = 0; v < V; v++) {
                    // If there is an edge between u and v
                    if (adj[u][v]) {
                        // If v is not colored, assign opposite color to u
                        if (color[v] == -1) {
                            color[v] = 1 - color[u];
                            queue[rear++] = v;
                        }
                        // If v is colored with same color as u, it's not bipartite
                        else if (color[v] == color[u]) {
                            return 0;
                        }
```

```
            }
        }
    }
  }
  }
  return 1;
}

int main() {
  int V, E;
  int adj[MAX][MAX] = {0};

  if (scanf("%d %d", &V, &E) != 2) return 0;

  for (int i = 0; i < E; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    // Undirected graph
    adj[u][v] = 1;
    adj[v][u] = 1;
  }

  if (isBipartite(adj, V)) {
    printf("Yes, the given graph is Bipartite\n");
  } else {
    printf("No, the given graph is not Bipartite\n");
  }

  return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

David is working on a project that involves traversing graphs for analyzing network connectivity. He is tasked with performing a Depth-First Search (DFS) traversal on a connected, undirected graph to explore all its vertices starting from a given node.

He needs help writing a program that can process multiple test cases where each test case describes a graph with its vertices and edges, and the program should output the DFS traversal order for each graph.

Can you assist David by writing a program that performs the DFS traversal for each test case?

Example

Input:

2

5 4

0 1 0 2 0 3 2 4

4 3

0 1 1 2 0 3

Output:

0 1 2 4 3

0 1 2 3

Explanation:

Visit all the vertices using a depth-first search algorithm using 0 as a source node.

Input Format

The first line of input contains an integer T denoting the number of test cases.

Each test case consists of two lines.

- The first line of each test case contains two integers N and E which denote the number of vertices and number of edges respectively.
- The second line of each test case contains E space-separated pairs u and v denoting that there is an edge from u to v.

Output Format

For each test case, the output displays the graph vertices in the depth-first

traversal order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2
5 4
0 1 0 2 0 3 2 4
4 3
0 1 1 2 0 3
Output: 0 1 2 4 3
0 1 2 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Structure for Adjacency List Node
struct Node {
    int vertex;
    struct Node* next;
};

// Structure for the Graph
struct Graph {
    int numVertices;
    struct Node** adjLists;
    int* visited;
};

// Function to create a node
struct Node* createNode(int v) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}
```

```
// Function to add edge (Undirected)
// Note: To match sample DFS order, we add nodes to the end of the list
// or maintain order to ensure consistency with the expected output.
void addEdge(struct Graph* graph, int s, int d) {
    struct Node* newNode = createNode(d);
    if (graph->adjLists[s] == NULL) {
        graph->adjLists[s] = newNode;
    } else {
        struct Node* temp = graph->adjLists[s];
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }

    newNode = createNode(s);
    if (graph->adjLists[d] == NULL) {
        graph->adjLists[d] = newNode;
    } else {
        struct Node* temp = graph->adjLists[d];
        while (temp->next) temp = temp->next;
        temp->next = newNode;
    }
}

// DFS Recursive Function
void DFS(struct Graph* graph, int vertex) {
    graph->visited[vertex] = 1;
    printf("%d ", vertex);

    struct Node* temp = graph->adjLists[vertex];

    while (temp) {
        int connectedVertex = temp->vertex;
        if (graph->visited[connectedVertex] == 0) {
            DFS(graph, connectedVertex);
        }
        temp = temp->next;
    }
}

void solve() {
    int N, E;
    if (scanf("%d %d", &N, &E) != 2) return;
```

```
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->numVertices = N;
    graph->adjLists = malloc(N * sizeof(struct Node*));
    graph->visited = malloc(N * sizeof(int));

    for (int i = 0; i < N; i++) {
        graph->adjLists[i] = NULL;
        graph->visited[i] = 0;
    }

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(graph, u, v);
    }

    // Start DFS from vertex 0 as per the problem explanation
    DFS(graph, 0);
    printf("\n");

    // Clean up memory for the next test case
    for (int i = 0; i < N; i++) {
        struct Node* temp = graph->adjLists[i];
        while (temp) {
            struct Node* toFree = temp;
            temp = temp->next;
            free(toFree);
        }
    }
    free(graph->adjLists);
    free(graph->visited);
    free(graph);
}

int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;
    while (T--) {
        solve();
    }
    return 0;
```

}

Status : Correct *Marks : 10/10*